

Project Synapse: Org-Wide Zero-ETL Semantic Data Core

COMPLETE INTEGRATED ENGINEERING SPECIFICATION DOCUMENT

0. Abstract & Architectural Soundness Verification

Project Synapse establishes architectural feasibility for an organization-wide, discrepancy-tolerant, schema-on-read semantic data framework. This system bypasses rigid, monolithic schema constraints, demonstrating completeness by resolving deep runtime vulnerabilities—including deterministic edge cases, cost parameters, latency barriers, and identity boundaries—using rigorous mathematical specifications and composable open-source engine components.

1. The Strategic Architecture & Vision at Scale

1.1 Problem Statement: The Data Transformation Bottleneck

Modern transaction engines yield distributed, semi-structured, and highly heterogeneous workloads (OLTP). To make this information actionable for analytical pipelines, enterprise frameworks historically impose a strict **Schema-on-Write** model. This mechanism demands costly upstream ETL pipelines (e.g., dbt, Apache Spark) to pull, filter, clean, and aggressively restructure raw operational metrics before they enter an optimized warehouse.

This operational model forces artificial, premature agreements across teams on target schemas. Minor upstream software mutations break downstream infrastructure, massive operational latency limits real-time access, and heavy cleaning steps eliminate vital context, native anomalies, and conversational nuances required by contextual reasoning engines.

The Strategic Synapse Core Thesis:

Retain raw, versioned source streams immutably inside an unorganized landing core. Defer data modeling and structural layout logic entirely to the query window via a dynamic Schema-on-Read processing engine. Maintain an active, temporal knowledge core mapping entities, assertions, and discrepancies as first-class citizens rather than fabricating a singular cleaned database view.

1.2 Core Architectural Axioms

- **Raw Immutability:** The landing repository is read-only; downstream engines never alter source payloads.
- **Lineage Tracking:** Every claim references exact source segments, code blocks, extraction runs, and timestamps.
- **Discrepancy Integrity:** Competing truths are fully preserved and mapped rather than being silently overridden.
- **Tiered Infrastructure:** Lightweight local cross-encoders evaluate basic paths by default; massive reasoning engines execute selectively.

- **Security Propagation:** Right-to-access matrices travel natively with the text blocks down to any derived nodes.

1.3 Scalability Targets for Engineering Mapping

Scale Vector	Architecture Reference Constraint
Corporate Domains	10–50 business domains (CRM, billing, infrastructure, legal logs, support).
Source Nodes	50–500 operational integrations and streaming watermarks.
Dataset Bounds	10^8 – 10^{11} objects or streaming log markers over the retention window.
Latency Classes	Hot path (seconds), warm interactive path (tens of seconds), deep asynchronous processing jobs.

2. Deep-Source Framework Components

Project Synapse removes conventional ETL constraints by orchestrating four distinct open-source core modules:

1. Graphiti (by Zep)

`github.com/getzep/graphiti`

Role: Continuous Context & Temporal Relation Modeler.

Functions via a Learned Ontology model, continuously ingesting data segments to map entities, attributes, and changing temporal states over time without manual database declarations.

Solves isolated context limits; maps temporal transitions across distributed log layers automatically.

2. GraphRAG (by Microsoft Research)

`github.com/microsoft/graphrag`

Role: Hierarchical Abstractor & Global Query Synthesizer.

Groups raw text structures into contextual community maps, enabling macro-level abstractions over wide corpora where localized vector index calls fail.

Manages macro analysis blockers; outputs cross-domain semantic overviews without fixed data warehousing schemas.

3. Data-Juicer (by Alibaba Tongyi Lab)

github.com/datajuicer/data-juicer

Role: Multi-Format Ingestion Pipeline & Raw Data Streamliner.

Provides a matrix of sandboxed computational operators to normalize, strip PII, and convert chaotic payloads into structured text lines ready for model context engines.

Manages byte-level data ingestion; cleans and standardizes logs on the fly without enforcing rigid endpoint tables.

4. PageIndex (by VectifyAI)

github.com/VectifyAI/PageIndex

Role: Reasoning-Based Document & Structural Tree Navigator.

Bypasses costly dense vector infrastructure scaling by building visual, layout-aware coordinate map trees directly on source text, steering models straight to precise document paths.

Mitigates dense vector monoculture scaling; performs precise leaf targeting straight over raw documents.

3. Structural Subsystem Integration Layer

Beyond isolated ingestion and embedding engines, enterprise production scalability requires an integrated control plane to govern cross-system transactions:

- **Connector & CDC Fabric:** Implements continuous write-ahead ingestion logging, applying rate limit thresholds and automated structural shift monitors.
- **Entity Resolution Engine (ER):** Uses blocking functions and cluster calculations to resolve identical real-world nodes (e.g., users, products) across separate domains.
- **Ontology Registry:** Tracks tiered schemas, dividing them into universal definitions (L0), domain specific packages (L1), and local team experimental schemas (L2).
- **Query Router & Budget Governor:** Evaluates inbound user queries to allocate computing tasks relative to explicit latency and financial envelopes.

4. Control Plane: Mathematical Guardrails & Custom Specifications

4.1 Information Density Routing Optimization

To prevent large context query pathways from causing cost scaling or server timeouts, the Query Router dynamically scores retrieved tokens using an internal Information Density Factor (IDF). The optimization engine minimizes total costs (\$V_c\$) and processing latency (\$V_l\$) subject to an absolute confidence floor (\$T\$):

$$\text{Minimize: } \alpha V_c + \beta V_l \quad \text{subject to} \quad C_f \geq T$$

The runtime Information Density Factor (IDF) evaluates how many valid relationships are packed into the text payload:

$$IDF = (\text{Count of Evaluated Predicates}) / (\text{Total Token Count})$$

- **High Density (IDF ≥ 0.75):** Bypasses large reasoning systems entirely, executing via fast cross-encoder reranking arrays.
- **Low Density (IDF < 0.25):** Routed through PageIndex visual coordinate trees, isolating token usage to highly targeted text zones.

4.2 Multi-Vector Conflict Resolution Matrix

Project Synapse does not purge discrepant inputs on write. At the query layer, it computes a dynamic Validity Weight (\$W_v\$) using designated Authority Rank (\$A_r\$), Lineage Proximity (\$L_p\$), and a Temporal Decay coefficient (\$\lambda\$) over elapsed time (\$\Delta t\$):

$$W_v = (A_r \times e^{-\lambda \cdot \Delta t}) + L_p$$

Conflict Typology	Technical Core Challenge	System Behavior
Scalar Clash	Opposing numeric metrics reported by distinct operational nodes within the same temporal window.	Flags as AMBIGUOUS_CONFLICT; outputs a split view mapping both parameters with source links if no explicit human_pin exists.
Temporal Supersession	Downstream streaming watermarks reporting conflicting values over an identical object instance.	If the newer event scores a higher \$W_v\$, the current view shifts state while maintaining the past state as a historic edge.
Domain Overlap	Opposing attribute states claimed across cross-functional domain barriers (e.g., CRM vs Billing name fields).	Resolves via primary authority maps inside the Ontology Registry; labels alternative fields as hidden aliases.

4.3 Cryptographic Attribute Propagation

Unstructured storage blocks carry an immutable Attribute-Based Access Control (ABAC) manifest block. When summaries or facts are derived by reasoning nodes, the engine applies a mathematical intersection of source rights:

$$\text{Derived Fact Key} = ACL_{\text{Object 1}} \cap ACL_{\text{Object 2}} \cap \dots \cap ACL_{\text{Object n}}$$

If a user's access rights clear the Intersected ACL Signature, the node compiles into the reasoning prompt; otherwise, it is entirely omitted from memory before execution.

5. Core Data Contract Specifications

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "SynapseDataCore",
  "defs": {
    "RawObject": {
      "type": "object",
      "required": ["object_id", "source_system", "content_hash", "ingested_at", "bytes_ref", "acl_tags"],
      "properties": {
        "object_id": { "type": "string", "format": "uuid" },
        "source_system": { "type": "string" },
        "content_hash": { "type": "string" },
        "ingested_at": { "type": "string", "format": "date-time" },
        "bytes_ref": { "type": "string", "format": "uri" },
        "acl_tags": { "type": "array", "items": { "type": "string" } }
      }
    },
    "Fact": {
      "type": "object",
      "required": ["fact_id", "subject_entity_id", "predicate", "object", "confidence", "evidence_refs"],
      "properties": {
        "fact_id": { "type": "string", "format": "uuid" },
        "subject_entity_id": { "type": "string", "format": "uuid" },
        "predicate": { "type": "string" },
        "object": { "type": "string" },
        "confidence": { "type": "number", "minimum": 0, "maximum": 1 },
        "evidence_refs": { "type": "array", "items": { "type": "string", "format": "uuid" } }
      }
    },
    "Conflict": {
      "type": "object",
      "required": ["conflict_id", "subject_entity_id", "predicate", "competing_fact_ids", "status"],
      "properties": {
        "conflict_id": { "type": "string", "format": "uuid" },
        "subject_entity_id": { "type": "string", "format": "uuid" },
        "predicate": { "type": "string" },
        "competing_fact_ids": { "type": "array", "items": { "type": "string", "format": "uuid" } },
        "status": { "type": "string", "enum": ["open", "resolved", "accepted_plural"] }
      }
    }
  }
}
```

6. Operational Target Domain: Infrastructure & Incident Mitigation

The designated prototyping playground for Project Synapse focuses on **Infrastructure & Operational Incidents**. This track presents a natural data environment filled with high discrepancy rates (e.g., CI deployment engines

reporting successful production distributions, while system performance monitors broadcast crash loops, and active engineers detail manual rollbacks inside troubleshooting chat rooms).

This testing ground proves that a schema-on-read context core can reconstruct operational incident paths without relying on monolithic data transformation layers.

7. Execution Roadmap & Sandbox Setup Steps

- **Phase 0 (Foundation Master):** Finalizing structural JSON contracts and locking down the core data definitions mapping.
- **Phase 1 (Ingestion Mock):** Writing a local Python worker to act as a Data-Juicer operator, scrubbing PII and adding cryptographic ACL tags before storing raw streams.
- **Phase 2 (Synaptic Memory Deployment):** Configuring a lightweight instance of Graphiti to parse raw objects and automatically extract temporal facts and conflict objects on read.
- **Phase 3 (Front-End Query Interface):** Deploying an operational web dashboard on Vercel connected to Google AI Studio API endpoints for real-time synthesis.